AI & ROBOTICS WORKSHOP

Appendix: Hands-On Notebook Walkthrough

Step-by-step instructions for connecting to JetBot and running the workshop notebooks, in learning order.

A1. What Is a Jupyter Notebook?

A Jupyter notebook is a document that mixes formatted text, live Python code, and interactive output in a single browser page. Each JetBot runs its own Jupyter server directly on the robot's onboard computer — your laptop or phone browser is simply the window into it. Nothing you click runs on your laptop; every command actually executes on the robot itself, which is why the robot must be powered on and reachable on the network before you can use it.

A notebook is organized into a vertical stack of independent blocks called cells. Every cell is one of three types:

Text Cells (Markdown)

- Formatted instructions, section headers, and explanations written in Markdown.
- Read-only in practice — you are not expected to run or edit these during the workshop, just read them before running the code cell(s) that follow.
- Use them as your guide: they tell you what the next code cell is about to do and why.

Code Cells

- Live, runnable Python. Click anywhere inside a code cell to select it, then press Ctrl+Enter to run just that cell or press Shift+Enter to run just that cell then automatically move to the next one.
- The bracket to the left of a cell shows its state: [ ] has not been run yet, [*] is currently running, and [3] (any number) has finished running — the number shows the order it was executed in.
- Cells share memory with each other. Variables and objects created in one cell (for example, the robot or camera object) remain available to every cell below it — which is why cells must be run top to bottom, in order, the first time through a notebook.
- If something behaves unexpectedly, it is usually because a cell was skipped or run out of order. When in doubt, use the Kernel menu (at top right) to restart and run all cells again from the top; select “Restart Kernel and Clear All Outputs …”.

Widgets

- Interactive controls — a live camera image, sliders, text boxes, and buttons — that appear directly beneath the code cell that creates them.
- Widgets are two-way: dragging a slider or clicking a button sends a value back to the robot in real time, and the robot's sensor data (like the camera feed) can drive what a widget displays.

- Because widgets update live, you can see the effect of a change immediately — no need to re-run a cell after moving a slider.

Why it matters: Every demo in today's workshop is built from this same pattern: a code cell wires up a camera or motor, and a widget gives you a live, interactive view of it.

A2. Opening Jupyter in Your Browser

Each JetBot runs its own Jupyter server and broadcasts its network address on the small OLED display mounted on top of the robot. Connect to it from any laptop, tablet, or phone that is on the same Wi-Fi network as the robot — no software installation is required.

Step 1 — Find the IP address

- Power on the JetBot and wait for it to finish booting (about 30–45 seconds).
- Read the IP address directly off the OLED display on top of the robot. It will look like 172.30.230.1xx.
- If the display is blank or shows no address, confirm the robot is connected to Wi-Fi, or ask your instructor for your unit's assigned address.

Step 2 — Open any browser

- Chrome, Firefox, Edge, and Safari all work equally well — use whatever is already on your laptop or phone.
- Make sure your laptop or phone is connected to the same Wi-Fi network as the JetBot; the notebook will not load from a different network.

Step 3 — Enter the address and port

- Type the IP address into the browser's address bar, followed by a colon and the port number 8888, which is the port Jupyter listens on.
- Press Enter. The Jupyter file browser should load within a few seconds, showing a list of notebook files on the robot.

```
http://172.30.230.1xx:8888
```

Note: If Jupyter asks for a password or token instead of loading the file browser directly, check with your instructor — workshop units are typically pre-configured to skip this step, but a freshly-flashed robot may still have it enabled.

Step 4 — Navigate the notebook folders

- Click the folder icon in the top-left corner of the JupyterLab page to open the file browser panel, if it isn't already showing.
- The file browser shows a list of folders, organized by topic. All of today's notebooks are inside the notebooks folder, which also contains the utilities, collision_avoidance and road_following subfolders.
- Double-click a folder to open it and see the notebooks inside, rather than searching through one long list of files.
- Use the table below to find which folder each of today's notebooks lives in.

Folder	Notebook(s) Inside
utilities	jupyter-starter.ipynb jetbot-motion-test.ipynb jetbot-camera-test.ipynb jetbot-gamepad-test.ipynb
road_following	Road_Following_Gamepad_Data_Collection.ipynb Road_Following_Train_Model.ipynb Road_Following_AI_Live_Demo.ipynb Road_Following_noAI_Live_Demo.ipynb (optional)
collision_avoidance	Collision_Avoidance_Gamepad_Data_Collection.ipynb Collision_Avoidance_Train_Model.ipynb Collision_Avoidance_Live_Demo.ipynb

From inside the matching folder, click the .ipynb file to open it in a new tab. The notebooks are listed in the order they are introduced below. Both workshops begin with the same four notebooks (A3–A6); after that, Workshop 1 continues with the Road Following notebooks (A7, plus the optional A8) and Workshop 2 continues with the Collision Avoidance notebooks (A9).

A3. First Notebook — jupyter-starter.ipynb

Start here, before touching the robot. Jupyter-starter.ipynb is a practice notebook that does not use the camera or the motors at all, so it is the safest place to get comfortable with text cells, code cells, the [ ] status indicator, restarting the kernel, and widgets. Every cell describes itself, so you can learn by reading and running it from top to bottom.

Step 1 — Open it and read the text cells

- In the file browser, double-click the utilities folder, then double-click jupyter-starter.ipynb to open it in a new tab.
- The first cells are Markdown text cells showing formatted text: headers, bold and italic, lists, a table, links, and math. Each one starts by describing what it is, so read it, then double-click a text cell to see the raw Markdown behind it and press Shift+Enter to render it again.

Step 2 — Run the code cells and watch the [ ] indicator

- Click a code cell, then press Ctrl+Enter to run it and stay on that cell, or Shift+Enter to run it and move to the next one. The comments inside each code cell (lines beginning with #) explain what it does.
- Run the cell that pauses for two seconds and watch the bracket to its left: [ ] before it runs, [*] while it is working, then a number such as [3] once it finishes. Run it again and the number goes up, because it records the order in which cells were executed.
- Keep going down the notebook through the cells for variables, loops, functions, and a simple plot. The output appears directly beneath each cell.

Step 3 — Practice restarting the kernel

- Run the cell that stores demo_value, then the cell after it that prints it. It works because the value is held in the kernel's memory.

- Open the Kernel menu and choose Restart Kernel..., then run the print cell again without re-running the first one. You will see a NameError, because a restart wipes everything from memory. Re-run the first cell and it works again.
- Next, try "Restart Kernel and Clear All Outputs ..." to see the notebook screen reset as well. This is the same recovery step to use if any robot notebook behaves unexpectedly.

Step 4 — Try the widgets

- Run the slider cell and drag the slider, then run the next cell to read its value back into Python. Then try the interact() cell, the Click me button (watch its counter go up), and the dropdown and text box that update a label as you change them.
- These are the same kinds of widgets used in the robot notebooks, from motor-speed sliders to the live camera image. Nothing in this notebook moves the robot, so experiment freely.

Before you move on: Once the status indicator, the kernel restart, and the widgets all feel familiar, you are ready for the robot notebooks. Close and shut down this notebook from the Jupyter file browser so it is not left running.

A4. Second Notebook — jetbot-motion-test.ipynb

Next, open jetbot-motion-test.ipynb. It is intentionally the simplest notebook that drives the robot, which makes it the best place to apply what you practiced in jupyter-starter.ipynb to real hardware before anything else is layered on top.

Step 1 — Read the text cells

- Scroll through the Markdown cells at the top of the notebook before running anything. They explain what the notebook does and introduce the Robot object that represents JetBot in code.
- There is nothing to click or run in a text cell — just read it, then move to the code cell below it.

Step 2 — Run the code cells in order

- Click the first code cell (it imports the JetBot library and creates a robot object), then press Shift+Enter to run it and advance to the next cell.
- Repeat Shift+Enter down the notebook, watching the [ ] next to each cell change to [*] while it runs and then to a number once it finishes.
- If a cell throws an error, it is almost always because an earlier cell was skipped — scroll back up and confirm every cell above it has a number in its bracket.

Step 3 — Try the slider widgets

- A later cell creates a pair of slider widgets, one for the left motor and one for the right motor, and links them directly to the robot's motor speed.
- Drag a slider and watch the corresponding wheel respond immediately — no need to re-run the cell.
- Try small, gentle movements first. Keep the robot on a stand or in open floor space, since the wheels will turn as soon as a slider moves off zero.

Safety tip: Before moving any slider, make sure JetBot has clear space around it, or is elevated on a stand with its wheels off the ground. Set every slider back to zero (or re-run the robot.stop() cell, if present) before closing the notebook.

A5. Third Notebook — jetbot-camera-test.ipynb

Next, open jetbot-camera-test.ipynb. This introduces the live camera image widget — the same pattern used in every demo later today, from collision avoidance to road following.

Step 1 — Initialize the camera

- Run the setup cell that creates the Camera object. This cell wakes up the physical camera hardware and configures the image size and frame rate.
- This step can take a couple of seconds the first time — wait for the [*] to turn into a number before continuing.

Step 2 — View the live image widget

- The next cell creates an Image widget and links it to the camera feed, so the live video appears directly inline in the notebook — no separate window or app required.
- You should see a continuously updating image from JetBot's camera within the notebook page itself. If the image is black or blank, make sure the lens cap has been removed from the camera.

Step 3 — Confirm frame rate and focus

- Check that the image is smooth and updating in real time, not frozen or stuttering.
- If the image looks blurry, gently rotate the small lens ring on the camera module to adjust focus while watching the live widget.
- Confirm the picture looks correct before moving on — later notebooks assume the camera is already working.

> **Before you move on:** Only one notebook can use the camera at a time. Before opening the next notebook, stop this one's camera cell (or close and shut down this notebook from the Jupyter file browser) so the camera hardware is free.

A6. Fourth Notebook — jetbot-gamepad-test.ipynb

With the camera confirmed working, open jetbot-gamepad-test.ipynb to drive JetBot manually with a game controller — the same manual-driving skill reused later for collecting training data.

Step 1 — Pair the gamepad

- Plug a USB gamepad into your laptop (or pair a Bluetooth one) before running the notebook — the controller connects to your browser, not to the robot directly. Go to the gamepad tester website http://hardwaretester.com/gamepad and note the index number; if needed, edit the " … Controller(index=)" line in the first cell.
- Run the setup cell that creates a Controller widget. If it doesn't detect your gamepad, press a button on the controller first, then re-run the cell.

Step 2 — Map controls to motors

- Display the controller widget and move the joysticks — confirm the on-screen axis indicators move the way you expect before linking anything to the motors.
- Run the cell that links specific joystick axes to the robot's left and right motor speed. Double-check which axis drives forward/back and which drives left/right.

Step 3 — Practice teleoperation

- With the link active, drive JetBot slowly around an open area of floor to get a feel for the controls before the live demos later today.
- Start with small, slow movements — it's easy to send more speed than intended on a first try.

Control	Action
D-pad Up	Drive forward at current speed
D-pad Down	Drive backward at current speed
D-pad Left	Pivot turn left
D-pad Right	Pivot turn right
Left joystick (push/pull)	Left motor spins clockwise (push) / counter-clockwise (pull), at current speed
Right joystick (push/pull)	Right motor spins clockwise (push) / counter-clockwise (pull), at current speed
Right Bumper (RB)	Increase speed
Left Bumper (LB)	Decrease speed
X button	Emergency stop + reset speed

Safety tip: Always drive in a clear, open area away from table edges, cables, and other robots. Know which cell to re-run (or which button stops the link) before you start driving, so you can stop quickly if needed.

A7. Workshop 1 — Road Following Notebooks

After the first four notebooks, Workshop 1 teaches JetBot to follow a road: you collect camera images while driving a track, train a model on them, then run it live. The notebooks are in the folder below. Run them in this order, with one optional extra at the end.

/jetbot/notebooks/road_following/

Step 1 — Collect the training data

- Open Road_Following_Gamepad_Data_Collection.ipynb from the folder above, pair the gamepad the same way as in A6, and run the cells in order.
- Set JetBot on the track and drive along it with the gamepad. Following the notebook's instructions, use the gamepad to move the green target dot on the live camera image to where JetBot should steer, and capture images as you go. Cover the whole track, including straights and curves.

Control	Action
D-pad Up	Drive forward at current speed
D-pad Down	Drive backward at current speed
D-pad Left	Pivot turn left
D-pad Right	Pivot turn right
Right joystick (X axis)	**Nudge target dot left / right (dot stays where you leave it)**
Right joystick (Y axis)	**Nudge target dot up / down (dot stays where you leave it)**
Right Bumper (RB)	Increase speed
Left Bumper (LB)	Decrease speed
A button	**Save current frame + (x, y) target to dataset_xy**
X button	Emergency stop + reset speed

- When you are done collecting, close and shut down this notebook from the Jupyter file browser so the camera is free for the next one.

Step 2 — Train the model

- Open Road_Following_Train_Model.ipynb from the same folder and run the cells in order. It uses the images you just collected to train the road following model.
- Training can take several minutes and runs on the robot itself. Wait for the [*] to turn into a number before moving on, and confirm the notebook finished without errors — the live demo loads the model that this notebook produces.
- Each workshop has its own training notebook (Workshop 2 uses Collision_Avoidance_Train_Model.ipynb), so make sure you opened the one in the road_following folder.

Step 3 — Run the AI live demo

- Open Road_Following_AI_Live_Demo.ipynb and run the cells in order. It loads your trained model and uses the live camera image to steer JetBot along the road.
- Set JetBot at the start of the track, start the demo using the notebook's widgets, and watch how closely it follows the road.

- If it wanders off the road, return to Step 1, collect more images (especially where it went wrong, such as the curves), and train again.

Safety tip: Run the live demo in a clear, open area away from table edges, cables, and other robots. Know which cell stops the robot (or re-run the robot.stop() cell, if present) before you start.

Optional: If time allows, continue to A8 to run Road_Following_noAI_Live_Demo.ipynb and compare the learned model with a hand-written, rules-based one.

A8. Optional — Road_Following_noAI_Live_Demo.ipynb

Workshop 1 can also run Road_Following_noAI_Live_Demo.ipynb, in the same road_following folder. This is a classic, rules-based computer-vision demo — no neural network or training involved — that sets up a direct contrast with the learned road-following model from the previous notebook.

Step 1 — Review the vision logic

- Before running the live loop, read through the code cells that locate the line: they convert each camera frame to a color mask, then find the center point of the line within that mask.
- There is no training or learning here — the thresholds and logic are hand-written and fixed, which is the key difference from the AI-based road-following model.

Step 2 — Run the live loop

- Set JetBot on a track with a clear, high-contrast line — dark tape on a light floor, or light tape on a dark floor, both work well.
- Run the control-loop cell. On each frame, it calculates how far the line's center is from the center of the image and nudges the motors to steer back toward it.

Step 3 — Compare to AI road following

- Watch for where this approach struggles — changes in lighting, shadows, reflections, or a line that curves sharply can all confuse a fixed color threshold.
- Use this as a discussion point: the AI road-following model from earlier in the workshop learns to handle this same task from examples, rather than from a hand-tuned threshold, which is why it tends to generalize better to conditions it hasn't seen before.

A9. Workshop 2 — Collision Avoidance Notebooks

After the first four notebooks, Workshop 2 builds a collision avoidance behavior from scratch: you collect your own camera images, train a model on them, then run it live so JetBot steers away from obstacles. All three notebooks are in the folder below. Run them in this order.

/jetbot/notebooks/collision_avoidance/

Step 1 — Collect the training data

- Open Collision_Avoidance_Gamepad_Data_Collection.ipynb from the folder above, pair the gamepad the same way as in A6, and run the cells in order.
- Drive JetBot with the gamepad and follow the notebook's instructions to save camera images of the two situations the model must tell apart: a clear path ahead, and an obstacle in the way. Collect a good variety — different obstacles, distances, angles, and lighting.

Control	Action
D-pad Up	Drive forward at current speed
D-pad Down	Drive backward at current speed
D-pad Left	Pivot turn left
D-pad Right	Pivot turn right
Left joystick (push/pull)	Left motor spins clockwise (push) / counter-clockwise (pull), at current speed
Right joystick (push/pull)	Right motor spins clockwise (push) / counter-clockwise (pull), at current speed
Right Bumper (RB)	Increase speed
Left Bumper (LB)	Decrease speed
A button	**Save current frame to dataset/free**
B button	**Save current frame to dataset/blocked**
X button	Emergency stop + reset speed

- When you are done collecting, close and shut down this notebook from the Jupyter file browser so the camera is free for the next one.

Step 2 — Train the model

- Open Collision_Avoidance_Train_Model.ipynb from the same folder and run the cells in order. It uses the images you just collected to train the collision avoidance model.
- Training can take several minutes and runs on the robot itself. Wait for the [*] to turn into a number before moving on, and confirm the notebook finished without errors — the live demo loads the model that this notebook produces.
- Each workshop has its own training notebook (Workshop 1 uses Road_Following_Train_Model.ipynb), so make sure you opened the one in the collision_avoidance folder.

Step 3 — Run the live demo

- Open Collision_Avoidance_Live_Demo.ipynb and run the cells in order. It loads your trained model and uses the live camera image widget, the same pattern from A5, to decide whether the way ahead is clear.

- Place JetBot on open floor with a few obstacles such as boxes or books, start the demo using the notebook's widgets, and watch how it responds.
- If it drives into obstacles, or turns away when nothing is there, return to Step 1, collect more varied images, and train again. A better dataset is usually the fix.

> **Safety tip:** Run the live demo in a clear, open area away from table edges, cables, and other robots. Know which cell stops the robot (or re-run the robot.stop() cell, if present) before you start.

A10. Notebook Sequence Recap

Both workshops start with the same four notebooks, each adding exactly one new skill on top of the last. After that, each workshop follows its own sequence. Run them in this order:

Both workshops — run these first

#	Notebook	Skill Learned
1	jupyter-starter.ipynb (folder: utilities)	Learn the notebook itself, with no robot motion: text cells, code cells, the [ ] status indicator, restarting the kernel, and widgets.
2	jetbot-motion-test.ipynb (folder: utilities)	Apply those basics to the robot: run code cells in order and drive the wheels with slider widgets.
3	jetbot-camera-test.ipynb (folder: utilities)	Add a live camera image widget — confirm the camera hardware and video feed are working.
4	jetbot-gamepad-test.ipynb (folder: utilities)	Add manual control — drive JetBot with a gamepad, the same skill used later for data collection.

Workshop 1 — Road Following

#	Notebook	Skill Learned
5	Road_Following_Gamepad_Data_Collection.ipynb (folder: road_following)	Drive the track with the gamepad to collect the camera images that teach JetBot to follow a road.
6	Road_Following_Train_Model.ipynb (folder: road_following)	Train the road following model from the images you collected.
7	Road_Following_AI_Live_Demo.ipynb (folder: road_following)	Run the trained model live: JetBot follows the road using what it learned from your examples.
opt.	Road_Following_noAI_Live_Demo.ipynb (folder: road_following)	Optional: see a full non-AI control loop run live, as a baseline to contrast with the learned model.

Workshop 2 — Collision Avoidance

#	Notebook	Skill Learned
5	Collision_Avoidance_Gamepad_Data_Collection.ipynb (folder: collision_avoidance)	Collect camera images with the gamepad that teach JetBot what an obstacle looks like.
6	Collision_Avoidance_Train_Model.ipynb (folder: collision_avoidance)	Train the collision avoidance model from the images you collected.
7	Collision_Avoidance_Live_Demo.ipynb (folder: collision_avoidance)	Run the trained model live: JetBot uses its camera to detect obstacles and steer away.

By the end of the first four notebooks you will have used every basic control pattern: reading and running a notebook, driving the wheels, viewing a live camera widget, and driving manually with a gamepad. The workshop notebooks then put those building blocks to work: collecting your own data, training a model, and running it live on JetBot.